Heuristiques et métaheuristiques pour l’ordonnancement sur machines parallèles non identiques

Projet Java

Eleves: Salim Kisma - Samy Taleb

Présentation de l’article

Article scientifique étudié:

« Efficient heuristics and metaheuristics for the unrelated parallel machine scheduling problem with release dates and setup times »

Athmani Mohamed, Arbaoui Boubekeur, Mimene Kaouthar, Yalaoui Farouk (2022)

Présentation du problème

Objectif général

- Implémenter et analyser des heuristiques et métaheuristiques visant à résoudre un problème d'ordonnancement industriel

Problème étudié :

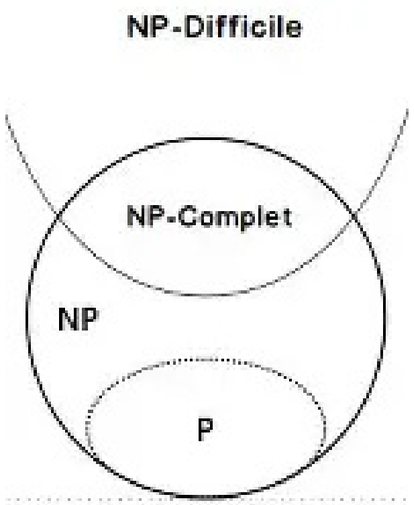
$$R \mid r_j,\ s_{ijk} \mid C_{\max}$$

Caractéristiques principales :

- **n** tâches à exécuter
- **m** machines parallèles non identiques (durées différentes par machine)
- contraintes de :
  - une date de disponibilité **rj** avant laquelle elle ne peut commencer,
  - un temps de setup **sijk** lorsqu'elle est exécutée après la tâche i sur la machine k.
  - une durée d'exécution **pjk** de la tâche j sur la machine k
- **objectif :** minimiser la durée totale (makespan)

$$\min C_{\max} = \min \max_{k \in M} \{C_k\}$$

Le problème est **NP-difficile**, même dans des versions simplifiées:
→ Impossible à résoudre de manière exacte sur des **tailles réalistes** et nécessité d'utiliser des **procédures heuristiques**.



$$P \neq NP$$

Modélisation conceptuelle et mathématique

Variables du modèle

- **pjk** : durée d'exécution de la tâche *j* sur la machine *k*
- **rj** : date de disponibilité de la tâche *j*
- **sijk** : temps de setup entre les tâches $i \rightarrow j$ sur la machine *k*
- **xjk** = 1 si la tâche *j* est affectée à la machine *k*, 0 sinon

Contraintes

$$\sum_{k=1}^{m} x_{jk} = 1 \quad \forall j$$

> Chaque machine exécute **une seule tâche à la fois** (pas de chevauchement, respect des setups).

Fonction objectif

Minimiser le **makespan** :

$$\min C_{\max} = \min \left(\max_{k \in M} C_k \right)$$

où **Ck** = date de fin de la dernière tâche sur la machine k.

Méthodes et algorithmes proposés

Heuristique BIBA (Best Insertion Based Approach)

Objectif :
Obtenir rapidement une solution initiale faisable et de bonne qualité pour lancer la méta-heuristique.

Principe général :

- Repose sur une logique gloutonne (greedy) :
- Parmi les tâches non planifiées, on choisit celle qui augmente le moins le makespan si on l'ajoute maintenant.
- La tâche choisie est insérée sur la machine où son ajout est le plus favorable
- On répète ce processus jusqu'à ce que toutes les tâches soient planifiées.

Intérêt de BIBA :

- Construction d'une solution initiale très rapide
- Approche simple mais efficace
- Base solide pour les méthodes d'amélioration (local search, métaheuristiques...)

Méthodes et algorithmes proposés

Recherche locale VND (Variable Neighborhood Descent)

Objectif :
Améliorer la solution initiale en explorant progressivement plusieurs voisinages, afin d'obtenir un optimum local plus performant avant d’appliquer la métaheuristique.

Principe général :

- Sur la solution courante,
- on explore un voisinage (échange, insertion, déplacement…).
- Si une amélioration du makespan est trouvée,
  - on accepte cette nouvelle solution
  - et on reste dans ce même voisinage.
- Si aucune amélioration n’est possible,
  - on passe au voisinage suivant.
- Le processus s'arrête lorsque tous les voisinages ont été testés sans aucune amélioration.

Intérêt :

- Amélioration significative de la solution de départ
- Exploration structurée et efficace des modifications possibles
- Converge vers un optimum local robuste
- Prépare une base de qualité pour les métaheuristiques

Méthodes et algorithmes proposés

Métaheuristiques globales

Problème :
La recherche locale peut rester bloquée dans un minimum local : mauvaise solution finale.

→ Solution
Utiliser des métaheuristiques capables d'explorer au-delà des optimums locaux.

1. Late Acceptance Hill Climbing (LAHC)

On compare la nouvelle solution non pas uniquement à la solution courante, mais à celle rencontrée L itérations plus tôt.
La nouvelle solution est acceptée si elle n'est pas pire que cette solution "ancienne".

2. Simulated Annealing (SA)

Inspiré du procédé physique de recuit simulé.
Une solution moins bonne peut être acceptée avec une probabilité dépendant : de la dégradation de la solution, d'une température T, qui décroît au fil des itérations.

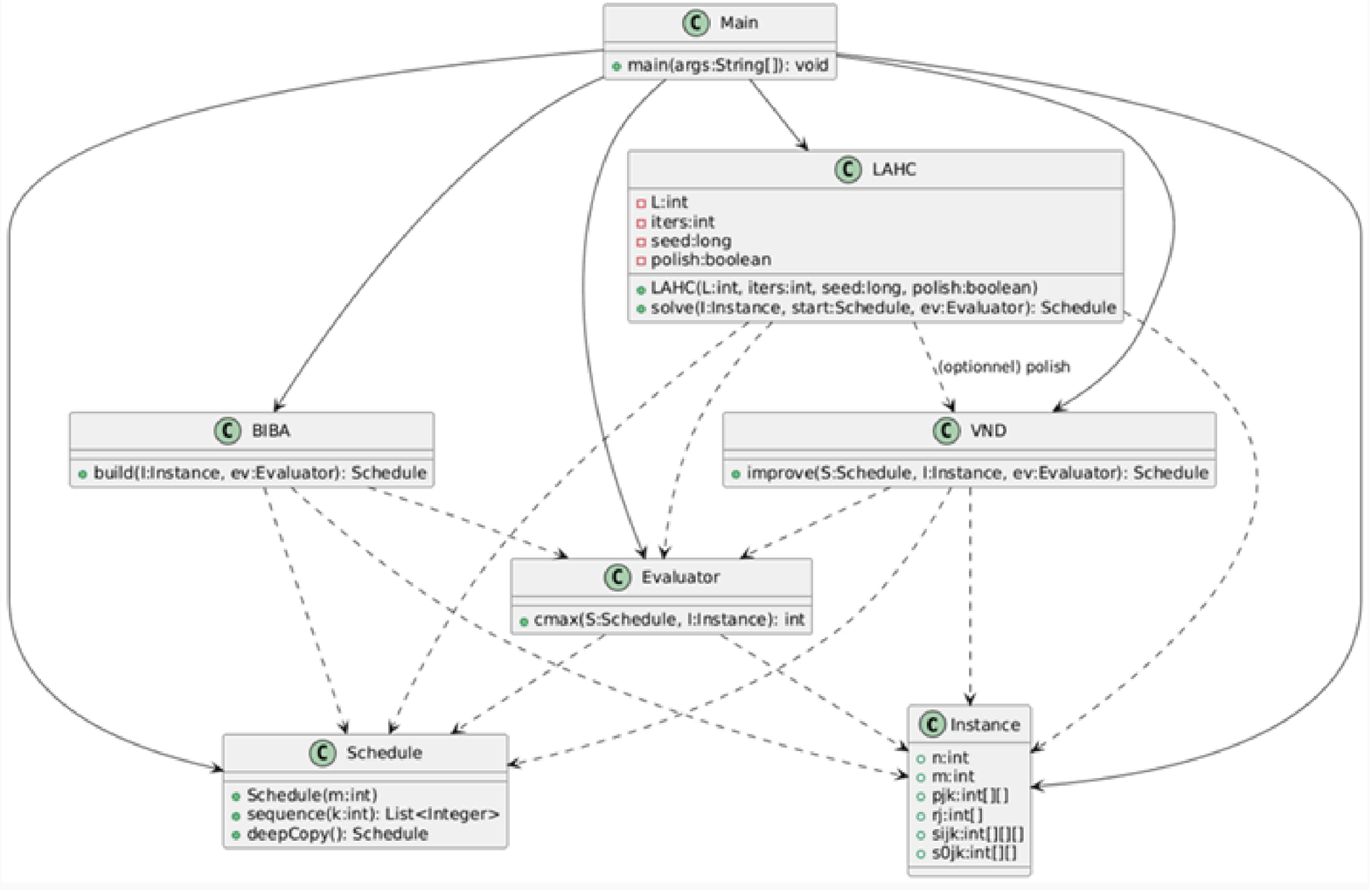
Résultats de l’article

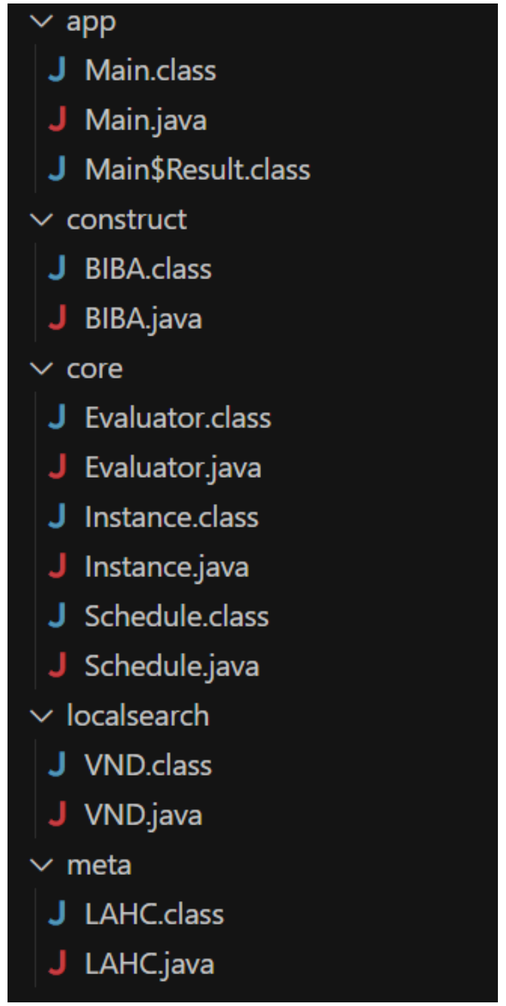
Enseignement des expérimentations :

Les résultats montrent que :

- BIBA et VND (heuristiques seules)
  - fournissent des solutions correctes très rapidement
- Les métaheuristiques (LAHC, SA)
  - améliorent nettement la qualité des solutions obtenues par VND
- LAHC
  - offre le meilleur compromis entre performance, robustesse et temps de calcul

Conception logicielle





Instances de tests

I1, petite instance sans setup

$$n = 6,\ m = 2, \quad p = \begin{bmatrix} 6 & 7 \\ 5 & 5 \\ 7 & 6 \\ 4 & 8 \\ 9 & 3 \\ 6 & 6 \end{bmatrix}, \quad r = \begin{bmatrix} 0 & 2 & 0 & 3 & 1 & 0 \end{bmatrix}$$

Aucun setup : $s_{ijk} = 0$ et $s^0_{jk} = 0$.

I2, instance avec setup unitaire

$$n = 8,\ m = 3, \quad p = \begin{bmatrix} 3 & 9 & 6 \\ 4 & 4 & 7 \\ 5 & 6 & 5 \\ 7 & 8 & 4 \\ 2 & 2 & 9 \\ 9 & 3 & 6 \\ 4 & 5 & 5 \\ 6 & 4 & 7 \end{bmatrix}, \quad r = \begin{bmatrix} 0 & 0 & 1 & 2 & 0 & 3 & 1 & 2 \end{bmatrix}$$

Setup séquentiel : $s_{ijk} = 1$ si $i \neq j$, 0 sinon. Pas de setup de démarrage.

I3, instance plus grande et déséquilibrée

$n = 12,\ m = 3, \quad p_{jk} \in [2, 12]$ avec disparités par machine, $\quad r_j \in [0, 5], \quad s_{ijk} = 1_{\{i \neq j\}}$

Les valeurs détaillées sont générées de manière déterministe à partir d'une graine fixée.

Mesures relevées

Mesures relevées :
Pour chaque instance et pour chaque méthode, les indicateurs suivants sont collectés :

- le makespan final Cmax ;
- le temps d'exécution (en millisecondes) ;
- les séquences obtenues pour chaque machine (à des fins de vérification visuelle).

Instance	n	m	BIBA	VND	LAHC	Commentaire
I1	6	2	16	16	16	Instance simple, pas d'amélioration
I2	8	3	13	13	13	Setups unitaires, plan déjà équilibré
I3	12	3	25	22	22	Amélioration nette par VND/LAHC

$$\text{I1} : M_0 = [5, 0, 3],\ M_1 = [2, 4, 1].$$
$$\text{I2} : M_0 = [0, 6, 4],\ M_1 = [1, 7, 5],\ M_2 = [2, 3].$$
$$\text{I3} : M_0 = [5, 8, 6],\ M_1 = [3, 7, 4, 1, 2],\ M_2 = [0, 9, 10, 11].$$

Conclusion

Qualité des solutions :

- Toutes les instances sont faisables :
  - aucune tâche manquante ou en double
  - respect des dates rj
- Les séquences finales produites par LAHC sont cohérentes et optimisées.

Comparaison des méthodes :

- BIBA : solution initiale correcte mais parfois sous-optimale (I3).
- VND : n'aggrave jamais BIBA et améliore nettement l'instance I3 (25 → 22).
- LAHC : résultats stables (min = avg = max) et aussi bons que VND.

Interprétation générale:

- I1 : instance simple → aucune amélioration possible.
- I2 : setups unitaires → solution déjà équilibrée.
- I3 : instance difficile → gains significatifs grâce à VND + LAHC.